

Välkomna till ++!

- 2016: **Simon Lindholm** och **Johan Sannemo** (16 studenter)
- 2017: Simon Lindholm och **Andreas Wallström** (7 studenter)
- 2018: Simon Lindholm och **Love Brandfeldt** (8 studenter)
- 2019: Love och **Christoffer Nolander** (7 studenter)
- 2020: Love och **Rickard Mårtensson** (9 personer)
- 2021: **Anton Lilja** och Rickard (8 personer)
- 2022: Anton och **Morris Hansing** (8 personer)
- 2023: Anton och Morris Hansing (16 personer)
- 2024: Anton och **Ludvig Lindström** (19 personer)
- 2025: Ludvig och **Tommy Bergman** (10 personer)
- 2026: Tommy och **Mikael Kemstedt** (13 personer)

- Tidigare ++:are (2023)
- Nintendo 64, modding, reverse engineering, emulering
- Operativsystem & inbyggda system
- C, C++, assembler
- METAspexet, DEMON ⚡, METAdorerna

- Tidigare **+ to ++:are** (2024)
- FP, Rust, Zig
- Data Engineer
- High Performance / performance engineer

- Namn?
- Favoritspråk?
- Intressen?
- Förväntningar?

- **DD1337/PROGINDA:** Rust, läsa + skriva kod, samarbete
- **DD1338/ALGINDA:** Algoritmer och datastrukturer
- **DD1396/PALINDA:** Parallellprogrammering, go lang (speedrun!)
- **DD1349/PROJINDA:** Projektuppgift i större grupp (slowrun?)

- Slack-klon
- Kattis-klon
- Logisim-klon
- Kernel
- VM
- Spel / spelmotor
- VCS

- Övningar är obligatoriska
- Nya läxor (typ) varje vecka
- Ska var klar och uppladdad **innan** övningen
- AI-genererad kod förbjudet
- Möjlighet att byta grupp
- Möjlighet att göra regular-läxor
- GitHub-organisation
- Discord

- All inlämningar + närvaro: A :)
- Ett betygsteg lägre för varje saknad inlämning

- Systemnivåspråk
- Hög prestanda + deterministisk minnesanvändning
- Memory safe vid kompilering*
- Funktionellt typsystem
 - Algebraic datatypes
 - Traits
 - Generics

- Printa en string!

```
fn print_string(s: String) {  
    println!("{}", s);  
}  
  
pub fn main() {  
    let s = "Hej!".to_string();  
    print_string(s);  
}
```

- Gör det igen!

```
fn print_string(s: String) {  
    println!("{}", s);  
}  
  
pub fn main() {  
    let s = "Hej!".to_string();  
    print_string(s);  
    print_string(s);  
}
```

- Ånej...

```
$ NO_COLOR=1 rustc example.rs
error[E0382]: use of moved value: `s`
  --> example.rs:8:18
   |
6  |         let s = "Hej!".to_string();
   |         - move occurs because `s` has type `String`, which does
not implement the `Copy` trait
7  |         print_string(s);
   |                     - value moved here
8  |         print_string(s);
   |                     ^ value used here after move
```

- Referenser!

```
fn print_string(s: &String) {  
    println!("{}", s);  
}  
  
pub fn main() {  
    let s = "Hej!".to_string();  
    print_string(&s);  
    print_string(&s);  
}
```

- Om man vill ändra ett värde med borrowing

```
pub fn main() {  
    let mut s = "Hej!".to_string();  
    let t = &mut s;  
    *t += " Hej igen!";  
    println!("{}", *t);  
}
```

Det går att ha:

- **Godtyckligt många vanliga borrows till ett värde, eller**
- **Exakt en muterbar borrow till ett värde**

...men aldrig samtidigt!

- ADT (sumtype), “en av”
- Olika strukturer

```
enum Event {  
    Tick,  
    KeyPress {key: i8, hold: bool},  
    KeyRelease(i8),  
    MouseMove([usize; 2], i8),  
}
```

```
fn print_enum(e: &Event) {  
    match e {  
        Event::Tick => println!("clock ticked"),  
        Event::KeyPress {key, hold} => println!(  
            "Pressed key: '{key}', is holding: {}",  
            hold  
        ),  
        _ => println!("Something else happened"),  
    }  
}
```

```
pub fn main() {  
    let e = Event::Tick;  
    print_enum(&e);  
    let e = Event::KeyPress {key: 1, hold: true};  
    print_enum(&e);  
    let e = Event::KeyRelease(1);  
    print_enum(&e);  
}
```

- Output:

```
$ cargo run  
clock ticked  
Pressed key: '1', is holding: true  
Something else happened
```

- Motsvarar typ NULL i andra språk
- Kan användas när det inte alltid finns ett värde
- T.ex. hitta index för elementet x i en lista

```
enum Option<T> {  
    None,  
    Some(T),  
}
```

- Används konsekvent för felhantering
- Kan innehålla ett “lyckat” värde eller information om vad som gick fel

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

- Skapa ett schackbibliotek i Rust
- Vi håller på med denna läxa i två veckor
- Fundera över hur biblioteket ska användas
 - (Nästa läxa är att skriva ett GUI utifrån en kamrats bibliotek)
- Tester/testprogram?
- Några länkar:
 - [Rustboken](#) / [Interaktiva rustboken](#)
 - [Chess programming wiki](#)
 - [Skapa bibliotek med Cargo](#)

- **Första veckan:** Fokusera på representation för bräde och pjäser, implementera vanliga drag.
- **Andra veckan:** Implementera schack + matt och specialdrag: en passant, rockad...

- Skriv GitHub-användarnamn på Discord
- Byt namn i servern till <riktigt namn> + <kth id>
- Mejla/säg till om nåt går snett

